ASSIGNMENT-17.1

Name: porika manikanta

HT.No: 2303A51271 **Batch:** 8

Task 1: Customer Data Cleaning

Scenario: In this task, AI-assisted Python scripting is used to clean a customer dataset. The script handles missing values, removes duplicates, standardizes text fields, and encodes categorical variables using pandas and scikit-learn.

Prompt: # Use AI to generate a Python script for cleaning a customer dataset **Instructions:**

- Handle missing values in columns (age, income, city).
- Remove duplicate records.
- Standardize text values (e.g., city names in lowercase).
- Encode categorical variables (gender, city). **Code:**

```
import pandas as pd from
sklearn.preprocessing import LabelEncoder

data = pd.read_csv('shopping_trends.csv')
print('=== Original Data ===')
print(data.head())

# Handle missing values for col in ['Age', 'Purchase Amount
(USD)', 'Review Rating']:     if col in data.columns:
data[col].fillna(data[col].median(), inplace=True)

# Remove duplicates
data.drop_duplicates(inplace=True)

# Standardize text columns to lowercase for col in
['Location', 'Category', 'Season']:     if col in
data.columns:         data[col] =
data[col].str.strip().str.lower()

# Label encode categorical variables
le = LabelEncoder() for col in ['Gender', 'Location',
'Category', 'Season']:     if col in data.columns:
data[col] = le.fit_transform(data[col].astype(str))

print('=== Cleaned Data ===')
print(data.head())
data.to_csv('cleaned_customer_data.csv', index=False)
```

Result:

Cleaned DataFrame with no missing values, no duplicates, standardized city names in lowercase, and all categorical columns label-encoded and ready for analysis.

Observation:

The AI-generated script accurately addressed all cleaning tasks. Missing values were filled using median imputation for numeric columns and mode for categorical ones. Duplicate rows were dropped. Text columns like Location and Category were standardized to lowercase. Label encoding converted categorical variables to numeric form, making the dataset ready for machine learning pipelines.

Task 2: Sales Data Preprocessing

Scenario: This task focuses on preprocessing sales transaction data using AI assistance. Date columns are parsed, new time-based features are engineered, the sales column is normalized, and outliers are detected and handled via IQR.

Prompt: # Preprocess sales transaction data using AI assistance **Instructions:**

- Convert date columns into datetime format.
- Create new features (month, year, day of week).
- Normalize sales amount column.
- Detect and handle outliers in dataset. **Code:**

```
import pandas as pd import numpy as np from
sklearn.preprocessing import MinMaxScaler

data = pd.read_csv('sales_data.csv')
print(data.head())

# Convert date column to datetime data['Date'] =
pd.to_datetime(data['Date'], errors='coerce')

# Extract time features
data['month']        = data['Date'].dt.month
data['year']         = data['Date'].dt.year
data['day_of_week']  = data['Date'].dt.dayofweek

# Normalize sales amount scaler = MinMaxScaler()
data['Sales_normalized'] =
scaler.fit_transform(data[['Sales']])

# IQR outlier capping
Q1, Q3 = data['Sales'].quantile(0.25), data['Sales'].quantile(0.75)
IQR = Q3 - Q1 data['Sales'] = data['Sales'].clip(Q1 -
1.5*IQR, Q3 + 1.5*IQR)

print(data.head())
data.to_csv('preprocessed_sales_data.csv', index=False)
```

Result:

A transformed dataset with parsed datetime, new columns for month/year/day_of_week, a normalized Sales_normalized column, and outliers capped using the IQR method.

Observation:

The AI script successfully converted date strings to datetime objects and extracted time-based features that are useful for trend analysis. The MinMaxScaler brought the sales column into the [0,1] range. IQR-based capping removed extreme outliers without discarding rows, preserving dataset size while improving distribution quality.

Task 3: Healthcare Data Preparation

Scenario: This task involves cleaning and preprocessing a healthcare dataset. Missing values in clinical columns are handled, categorical features are encoded, numerical features are scaled, and the dataset is split into training and testing sets for ML readiness.

Prompt: # Clean and preprocess a healthcare dataset using AI scripts **Instructions:**

- Handle missing values in blood_pressure, cholesterol.
- Encode categorical features.
- Scale numerical features (min-max or standard scaling).

- Split dataset into training and testing sets. **Code:**

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder,

StandardScaler from sklearn.model_selection import

train_test_split data =

pd.read_csv('healthcare_risk_factors.csv')

# Fill missing values
for col in ['blood_pressure', 'cholesterol',
'BloodPressure', 'Cholesterol']:     if col in data.columns:
data[col].fillna(data[col].median(), inplace=True)

for col in data.select_dtypes(include='object').columns:
data[col].fillna(data[col].mode()[0], inplace=True)

# Encode categorical features
le = LabelEncoder() for col in
data.select_dtypes(include='object').columns:
data[col] = le.fit_transform(data[col].astype(str))

# Standard scale numerical features
scaler = StandardScaler()
num_cols = data.select_dtypes(include=np.number).columns[:-1]
data[num_cols] = scaler.fit_transform(data[num_cols])

# Train-test split
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
print('Train:', X_train.shape, '  Test:', X_test.shape)
```

Result:

A fully preprocessed dataset with no missing values, all features encoded and standard-scaled, and an 80/20 train-test split yielding separate X_train, X_test, y_train, and y_test arrays ready for ML model training.

Observation:

The script handled missing clinical values using median imputation for robustness against skewed distributions. Label encoding converted all textual fields to numeric form. Standard scaling normalized feature magnitudes, which is critical for distance-based and gradient-descent algorithms. The 80/20 split provides sufficient training data while retaining a representative test set for unbiased evaluation.

Task 4: Employee Salary Data Preprocessing

Scenario: This task cleans and preprocesses an employee salary dataset using AI scripts. Missing values are filled, salary outliers are removed, department names are standardized to lowercase, and label encoding is applied to job location and department columns.

Prompt: # Clean and preprocess an employee salary dataset using AI scripts **Instructions:**

- Handle missing values.
- Detect and remove salary outliers.
- Standardize department names (lowercase).
- Apply label encoding to job location, department. **Code:**

```
import pandas as pd import numpy as np from
sklearn.preprocessing import LabelEncoder

data = pd.read_csv('employee_salary.csv')
print(data.head())

# Handle missing values for col in
data.select_dtypes(include=np.number).columns:
data[col].fillna(data[col].median(), inplace=True) for
col in data.select_dtypes(include='object').columns:
data[col].fillna(data[col].mode()[0], inplace=True)

# Remove salary outliers using IQR
Q1 = data['salary'].quantile(0.25)
Q3 = data['salary'].quantile(0.75)
IQR = Q3 - Q1
data = data[(data['salary'] >= Q1 - 1.5*IQR) &
(data['salary'] <= Q3 + 1.5*IQR)]

# Standardize department names to lowercase data['department']
= data['department'].str.strip().str.lower()

# Label encode department and job location
le = LabelEncoder() for col in ['department',
'job_location']:     if col in data.columns:
data[col] = le.fit_transform(data[col].astype(str))

print(data.head())
data.to_csv('preprocessed_employee_salary.csv', index=False)
```

Result:

A structured, clean dataset with no missing values, salary outliers removed, department names lowercase, and location/department label-encoded — ready for HR analytics and workforce modelling.

Observation:

The AI-generated script effectively handled all preprocessing steps for the HR dataset. Median and mode imputation preserved data volume while managing missing entries. IQR-based outlier removal eliminated extreme salary values that could skew compensation analysis. Lowercasing department names ensured consistency for grouping operations, and label encoding enabled department and location to be used as numeric inputs in analytical models.